

5、雷达警卫趣味玩法

该章节需要搭配小车底盘以及其它传感器才能运行，这里只是说明实现的方式，实际上运行不了，需要搭配本公司的RDK-X3旭日派小车实现该功能。要移植到自己的主板上运行，还需安装其它依赖，因此，这里只是做代码演示，望须知。

5.1、功能说明

程序启动后，小车会跟踪最近的一个点的目标，当目标点横向移动的时候，会跟目标点运动，当目标点靠近小车，小于设定的距离后，蜂鸣器会报警嘀嘀嘀响声，直到目标点远离小车设定的距离。

5.2、代码参考路径

该功能源码的位置位于配套虚拟机中，

```
~/oradar_ws/src/yahboomcar_laser/yahboomcar_laser/laser_warning.py
```

5.3、程序启动

终端输入，

```
ros2 launch yahboomcar_laser laser_warning_launch.py
```

5.4、核心代码解析

laser_warning_launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

import os
from ament_index_python.packages import get_package_share_directory
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource

def generate_launch_description():
    ms200_scan_node = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('oradar_lidar'), 'launch'),
            '/ms200_scan.launch.py'])
    )

    laser_warning_node = Node(
        package='yahboomcar_laser',
        executable='laser_warning',
    )

    bringup_node = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('yahboomcar_bringup'), 'launch'),
            '/yahboomcar_bringup_launch.py'])
    )
```

```

)

launch_description = LaunchDescription([
    ms200_scan_node,
    laser_warning_node,
    bringup_node
])
return launch_description

```

主要是启动雷达ms200_scan_node、底盘bringup_node和跟踪功能节点laser_warning_node。主要是看laser_warning.py文件。

laser_warning.py

主要看雷达的回调函数，这里说明了如何获取到每个角度的障碍物距离信息，然后获取最小距离的ID，计算角速度的大小，然后最小距离与设定的距离进行比较，如果小于设定的距离，则让蜂鸣器响，最后发布速度话题数据。

```

def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return
    if self.Joy_active == True or self.Switch == False:
        if self.Moving == True:
            self.pub_vel.publish(Twist())
            self.Moving = not self.Moving
        return

    self.Moving = True

    minDistList = []
    minDistIDList = []
    ranges = np.array(scan_data.ranges)
    for i in range(len(ranges)):
        angle = (scan_data.angle_min + scan_data.angle_increment * i) * 180
/ pi
        if angle > 180: angle = angle - 360
        if abs(angle) < self.LaserAngle and ranges[i] > 0:
            minDistList.append(ranges[i])
            minDistIDList.append(angle)
    if len(minDistIDList) != 0:
        minDist = min(minDistList)
        minDistID = minDistIDList[minDistList.index(minDist)]
    else:
        return

    velocity = Twist()
    angle_pid_compute = self.ang_pid.pid_compute(minDistID / 36, 0)
    if abs(angle_pid_compute) < 0.02:
        velocity.angular.z = 0.0
    else:
        velocity.angular.z = angle_pid_compute
    self.pub_vel.publish(velocity)

    if minDist <= self.ResponseDist:
        if self.Buzzer_state == False:
            b = Bool()

```

```
        b.data = True
        self.pub_Buzzer.publish(b)
        self.Buzzer_state = True
    else:
        if self.Buzzer_state == True:
            self.pub_Buzzer.publish(Bool())
            self.Buzzer_state = False
```